

Lectures Notes on Proof by Induction

Software Verification

Nova School of Science and Technology
Mário Pereira `mjp.pereira@fct.unl.pt`

Version of September 21, 2026

The contents of this document is strongly inspired by notes by Michael Erdmann and Frank Pfenning, and Bernardo Toninho.

1 Proof by Induction

The simplest form of induction, which you may recall from math or calculus classes, is induction over the naturals $0, 1, \dots$. This is known by a few names: *mathematical induction*, *standard induction*, *simple induction* or *weak induction*. The idea is simple but powerful: assume we want to prove a property for every natural number n . We first prove that the property holds for the first natural number, 0 (the base case). Then, we assume the property holds for an arbitrary natural n and establish it for the successor of n (i.e. $n+1$ – the inductive step). The two steps together guarantee the property holds for all natural numbers – why? We know it holds for 0 , and because of the inductive step, it must hold for its successor 1 . By the same account, if it holds for 1 then it must hold for its successor 2 , and so on, for any given natural number.

There are many small variations of this general scheme which can be easily justified and which we will also call mathematical induction. For instance, induction might start at 1 if we want to prove a property of all positive integers rather than the naturals.

As you may have guessed, proofs by induction are the main workhorse when reasoning formally about (recursive) functional programs. To see how it all works, consider the following recursive function:

```
let rec power (n: nat) (m: nat) : nat =                               OCaml
  match m with
  | 0 -> S 0
  | S m' -> mult n (power n m')
```

The function above (inefficiently) computes n^m for an arbitrary values n and m by multiplying n with itself m times. We take $0^0 = 1$.

Lets now prove that `power n m` does indeed calculate n^m , using induction over the naturals.

Theorem 1.1. `power n m` $\Downarrow n^m$ for all natural numbers m and n .

Proof. By mathematical induction on m .

- Base Case [$m = 0$]:

We need to show that $\text{power } n \ 0 \Downarrow n^0$, for all n .

$$\begin{aligned} & \text{power } n \ 0 \\ \Rightarrow^* & 1 \quad (\text{by simplification}) \end{aligned}$$

- Induction Step:

Assume that, for some $m \geq 0$, and all integers n , $\text{power } n \ m \Downarrow n^m$.
Prove that the property holds for $m + 1$.

We need to show that $\text{power } n \ (m + 1) \Downarrow n^{m+1}$.

$$\begin{aligned} & \text{power } n \ (m + 1) \\ \Rightarrow^* & n * \text{power } n \ (m + 1 - 1) \quad (\text{by simplification}) \\ \Rightarrow^* & n * \text{power } n \ m \quad (\text{by math}) \\ \Rightarrow^* & n * n^m \quad (\text{by the induction hypothesis}) \\ \Rightarrow & n \times n^m \quad (\text{by evaluation of } *) \\ = & n^{m+1} \quad (\text{by math}) \end{aligned}$$

which completes the proof.

□

The level of detail of a proof typically depends on the context in which the proof is carried out (and the mathematical sophistication of the expected reader). In this course, you should feel free to omit the number of computation steps and combine obvious reasoning steps. Appeals to the induction hypothesis and other non-obvious steps (like appealing to lemmas) should be justified as in the example above.

You should convince yourself that the reasoning above does indeed prove the power function is correct: we show that $\text{power } n \ 0$ is correct, for all n (base case); we show that, for some m and all n , if we assume $\text{power } n \ m$ is correct (the inductive hypothesis) then $\text{power } n \ (m + 1)$ is correct (inductive case). Then, we can carry out this reasoning for an arbitrary m : the function is correct for 0; by the inductive step, it is correct for 1; by the inductive step, it is correct for 2, and so on up-to m . This is often the intuitive reasoning one performs when writing recursive functions.

2 Generalizing the Induction Hypothesis

From the example above it may seem that proofs by induction are always straightforward. While this is the case for many proofs, there is the occasional function whose correctness proof turns out to be more difficult. This is often because the statement we are trying to prove is *too weak*, and what we have to prove is something more general than the final result we are aiming for. This is referred to as *generalizing the induction hypothesis*. A common symptom of the need to generalize the induction hypothesis (i.h.) is when an appeal to the i.h. results in a statement that is overly specialized and does not actually allow us to proceed with our reasoning.

While it can be shown that there is no general recipe (i.e., an algorithm) for this that will always work, we can isolate a common case. Consider the

following alternative implementation of the power function, through the use of a helper function:

```

let rec pow (n: nat) (m: nat) (r: nat) : nat = OCaml
  if eqb m 0 then r else pow n (m - 1) (r * n)

let power_alt (n: nat) (m: nat) : nat =
  pow n m 1

```

The `pow` function uses its third argument `r` as an *accumulator*, that is then used as the final result for `pow` when `m` is 0. You should convince yourself that the definition of `power_alt` above and the definition of `power` from before are indeed equivalent.

As an aside, this is a commonly used trick in functional programming. In the earlier `power` function, if we want to calculate `power 2 4` we must calculate `power 2 3` and multiply it by 2, which in turn requires calculating `power 2 2` and multiplying that intermediate result by 2, and so on. Compilers for functional languages must make arrangements for storing the intermediate computations, so that the final result can be correctly “unrolled” back. On the other hand, the `pow` function above does not have this kind of recursive structure – it features so-called *tail recursion*. For instance, to calculate `pow 2 4 1` we need only calculate `pow 2 3 2`, and so `pow 2 2 4`, and so `pow 2 1 8` and finally `pow 2 0 16`, which is 16. No “unrolling” back of intermediate results is needed. For functions of this shape, compilers can optimize away any special arrangements for storing intermediate computations, which can make the execution of tail-recursive functions such as `pow` much more efficient when compared to the execution of functions like `power`. This is called *tail-call optimization*.

We would now like to prove that $\text{power_alt } n \ m \Downarrow n^m$, for all naturals n and m . This requires proving something about `pow`, which takes one more argument. Intuitively, we would like to prove $\text{pow } n \ m \ 1 \Downarrow n^m$. Unfortunately, proving this statement directly by induction will fail. Lets see where it breaks down.

Suppose we have assumed the induction hypothesis:

$$\text{pow } n \ m \ 1 \Downarrow n^m$$

and try to prove

$$\text{pow } n \ (m + 1) \ 1 \Downarrow n^{m+1}$$

We proceed as we did in the inductive case for `power`:

$$\begin{aligned}
& \text{pow } n \ m + 1 \ 1 \\
\Rightarrow^* & \text{pow } n \ (m + 1 - 1) \ (1 * n) \\
\Rightarrow & \text{pow } n \ m \ (1 * n) \\
\Rightarrow & \text{pow } n \ m \ n
\end{aligned}$$

but now we are stuck. We cannot apply the induction hypothesis since the last argument in the call to `pow` is not 1. The solution is to generalize the property to allow any `r: nat` in such a way that the desired result follows easily. The generalization is as follows:

Lemma 2.1. $\text{pow } n \ m \ r \Downarrow r \times n^m$, for all r , all $n \geq 0$, and all m natural.

Proof. By mathematical induction on m .

Base Case: $m = 0$

We need to show that $\text{pow } n \ 0 \ r \Downarrow r \times n^0$, for all n .

$$\begin{aligned} & \text{pow } n \ 0 \ r \\ \Rightarrow^* & r && \text{(by simplification)} \\ = & r \times n^0 && \text{(by math)} \end{aligned}$$

Induction Step: Assume that, for some $m \geq 0$, and all integers n and r' , $\text{pow } n \ m \ r' \Downarrow r' \times n^m$. Prove that the property holds for $m + 1$.

We need to show that $\text{pow } n \ (m + 1) \ r \Downarrow r \times n^{m+1}$.

$$\begin{aligned} & \text{pow}(n, m + 1, r) \\ \Rightarrow^* & \text{pow } n \ (m + 1 - 1) \ (r * n) && \text{(by simplification)} \\ \Rightarrow^* & \text{pow } n \ m \ (r \times n) && \text{(by math and evaluation of *)} \\ \Rightarrow^* & (r \times n) \times n^m && \text{(by the induction hypothesis, with } r' = r \times n) \\ = & r \times n^{m+1} && \text{(by math)} \end{aligned}$$

which completes the proof.

□

The correctness of `power_alt` can now be established directly.

Lemma 2.2. $\text{power_alt } n \ m \Downarrow n^m$, for all natural numbers n and m .

Proof.

$$\begin{aligned} & \text{power_alt } n \ m \\ \Rightarrow & \text{pow } n \ m \ 1 && \text{(by simplification)} \\ \Rightarrow^* & 1 \times n^m && \text{(by Lemma 2.1)} \\ = & n^m && \text{(by math)} \end{aligned}$$

□

3 Beyond Natural Numbers

Up to this point we have seen how to reason precisely about functions that manipulate and produce (natural) numbers through the proof method of mathematical induction. However, as you may recall from any course you may have participated in that featured functional programming, most interesting or realistic recursive functions manipulate more sophisticated data structures such as lists and trees – themselves often defined as recursive (or inductive) data types.

Consider for instance the following piece of code which defines a inductive data type `list` of natural numbers, and a recursive function that adds all the elements of a list:

```
type list = Nil | Cons of nat * list OCaml

let rec add (l : list) : nat =
  match l with
  | Nil -> 0
  | Cons x xs -> plus x (add xs)
```

The type has exactly two constructors, `Nil` and `Cons`. The `Nil` constructor codifies the empty list. The `Cons` constructor represents a (singly-linked) cell containing an element of type `nat` and (a reference to) the rest of the list.

The `add` function takes an argument `l` of type `list` and produces a value of type `nat` which is the sum of all elements of `l`. It does so by pattern matching on the structure of the given list. Since `l` is of type `list` it can only be constructed using a `Nil` or a `Cons`. In the former case, it means we are trying to add the elements of the *empty* list and so the result must be `0`. In the latter case, we are dealing with a list with at least one element and a (possibly empty) tail. Pattern matching allows us to *name* the components of the `Cons` cell so that we may use them directly, thus we write `| Cons x xs -> plus x (add xs)` to denote that when the list is a `Cons`-cell carrying a natural value `x` and rest of the list `xs`, the `add` function is defined by adding `x` to the result of adding the rest of the list.

While it may be slightly harder to wrap your head around why, informally, the function `add` above is correct, intuitively we can see that it must produce the correct result: for empty lists it will produce `0`, for one element lists it will add that element to `0`, and so on. Moreover, since each call of `add` deconstructs one element of the list, and there can only be finitely many such elements, we can also convince ourselves that the function always terminates.

3.1 Structural Induction

We will now make the correctness argument precise (*i.e.*, formal) using a generalization of the technique from the previous sections: *structural induction*. The idea is that we can reason using induction on the possible *structure* of so-called inductive data types such as lists and trees in the sense given above. Instead of just postulating the induction principle over lists, we will attempt to construct it by analogy with the natural numbers.

Let us revisit the induction principle for natural numbers: to show that a property holds for all natural numbers we first prove that it holds for zero (*i.e.*, prove for `0`) and then, we assume the property holds for an arbitrary natural `n'` and prove the property holds for its successor (*i.e.*, assume property for `n'`, prove for `S n'`). The reason why this is a valid proof principle is the same as that for mathematical induction: values of type `nat` can only be either `0` or finitely generated by repeated application of `S` to a `nat`. Thus, for any given `nat`, if its `0`, the “base case” shows us the property holds; if its the successor of `0` (`S 0`, the inductive step together with the base case shows us the property holds; if its `S S 0`, the base case and two instances of the inductive step discharge the proof, and so on.

We can now reconstruct the induction principle for lists by following a similar recipe. We have as many cases as there are constructors in the data type: a base (or non-inductive case) for `Nil` and an inductive case for `Cons`. Spelling it out, to show a property holds for all values of type `list` we show the property holds for `Nil` and then, assuming it holds for an arbitrary list `l'`, we show that it holds for `Cons n l'`, for an arbitrary `n`. Let us show the `add` function over lists of integers is correct in the following sense:

Theorem 3.1. *For all $l : \text{list}$, $\text{add } l \Downarrow l_0 + \dots + l_n$, where l_i denotes the i -th element of the list, and the length of l given by n .*

Proof. By structural induction on l .

Case: l is `Nil`

We need to show that $\text{add}(\text{Nil}) \Downarrow 0$

$$\begin{aligned} & \text{add Nil} \\ \Rightarrow^* & 0 \quad (\text{by simplification}) \end{aligned}$$

Case: l is `Cons n l'`, for arbitrary n and l'

Assume that, $\text{add } l' \Downarrow l_0 + \dots l_m$ where the length of l' is m (*i.e.*, the induction hypothesis).

We need to show that $\text{add } (\text{Cons } n \ l') \Downarrow n + l_0 + \dots l_m$.

$$\begin{aligned} & \text{add } (\text{Cons } n \ l') \\ \Rightarrow^* & n + \text{add } l' \quad (\text{by simplification}) \\ \Rightarrow^* & n + (l_0 + \dots l_m) \quad (\text{by the induction hypothesis}) \\ = & n + l_0 + \dots l_m \quad (\text{by math}) \end{aligned}$$

which completes the proof.

□

3.2 Induction on Trees

Structural induction is by no means restricted to lists. In fact, one can reason by induction over any *recursively* defined data type. As mentioned before, when proceeding by induction, we have as many cases in our proofs as there are constructors in your type. Moreover, we have an induction hypothesis for recursive sub-component of a value.

Let us consider binary trees as our working ground. Such a data structure is defined in OCaml as follows:

```
type tree = Leaf | Node of tree * nat * tree OCaml
```

Now, we can define a simple function to manipulate values of type `tree`:

```
let rec mirror (t: tree) : tree = OCaml
  match t with
  | Leaf -> Leaf
  | Node (l, v, r) -> Node (mirror r, v, mirror l)
```

The above function simply takes the elements of a tree and, recursively, exchange them from left to right (and vice-versa), acting exactly as a mirror.

Now, time to prove properties about `mirror`. Consider the following Lemma:

Lemma 3.2 (MIRROR_INVOLUTIVE). $\forall t : \text{tree}, \text{mirror } (\text{mirror } t) = t$.

This statement tells the `mirror` function is an *involution* operator. In practice, when called twice, it acts as the identity function. Lets prove the lemma.

Proof. By structural induction on t .

- Base case [$t = \text{Leaf}$]: trivial (Homework: why?).
- Inductive step [$t = \text{Node } (l, v, r)$]:

- In this case, we have two induction hypotheses, one for each sub-tree:

$$\text{mirror } (\text{mirror } l) = l \quad (\dagger)$$

$$\text{mirror } (\text{mirror } r) = r \quad (\ddagger)$$

- We proceed the proof as follows:

$$\begin{aligned} & \text{mirror } (\text{mirror } t) = t \\ \Rightarrow^* & \text{Node } (\text{mirror } (\text{mirror } l), v, \text{mirror } (\text{mirror } r)) = t && \text{(by simplification)} \\ \Rightarrow & \text{Node } (l, v, \text{mirror } (\text{mirror } r)) = t && \text{(by rewriting with IH } \dagger) \\ \Rightarrow & \text{Node } (l, v, r) = t && \text{(by rewriting with IH } \ddagger) \end{aligned}$$

which completes the proof.

□